Over-reliance on Testing is Considered Harmful

In the quest to create high-quality software, most software companies spend enormous amounts of money and resources on testing. Is this a good strategy?

A few years back, I bought a costly smartphone, attracted by its 'cool' features. Once I started using it, I realised that it was way too buggy. Let me give you two examples. The mobile came with supporting software on a CD. After installing the software and starting it, it crashed, throwing a Visual C++ runtime error: "R6025: Pure virtual function call"! As a C++ expert, I could easily understand the bug: it is illegal to call a pure virtual function from a constructor, and when we make such a call, it will crash the application. But the question was: why hadn't the testing team for that mobile phone software caught it before releasing it?

Another strange problem with my mobile is that it 'freezes' or 'hangs' if I talk for 'too long'. After a few freezes, I figured out that 'too long' means approximately half-an-hour, which is not really that long! By freeze or hang I mean the screen would go blank, and it wouldn't respond to any key presses. It meant I couldn't restart the mobile (which required pressing keys). The only thing I could do was to remove the battery and put it back! Later, I also figured out a work-around; if I plugged the mobile to its charger, it would immediately spring back to life! This discovery showed me an important aspect of the problem—since it recovered when an event occurred (plugging in the charger, in this case), it is likely to be a software bug!

When I talked to my friends about this problem, most of them said they were not surprised—they said that bugs are very common in mobiles. Reading about the strategies of mobile phone companies and talking to my friends working in those companies was enlightening. Mobile phone companies face cut-throat competition and only those who first deliver the latest and coolest functionality to the market, survive! In this scenario, the main quality strategy to test the software is: if the tested features work, it goes to market! Maybe there is a bit of exaggeration in that statement, but it more or less captures the essence of how these companies focus on functionality, and how they use testing as the primary means to check quality.

The situation is not so different in the software industry, where testing is the main approach for improving software quality. According to Boris Beizer (a well-known researcher in testing), testing accounts for approximately half of the total software development costs! Depending on the size and type of the software company, the ratio of developers to testers ranges from 5:1 to 1:1! It is safe to say that software companies rely too much on testing.

Let us take a holistic view of testing, to understand why the focus on software testing is not the way to create high-quality software.

E W Dijkstra, the well-known Dutch computer scientist, once said, "Testing can show the presence of bugs, not their absence." This statement appears to be a clever play of words—but, if you think about it, it is insightful. What he means is, with testing, you can only check if the software has bugs. If you encounter no bugs while testing, it just means that testing did not uncover any bugs; however, you cannot say there are no bugs in the software. For this reason, by doing testing alone, you cannot say that software will work fine because it is unfeasible and impossible to test all the possibilities. This meaning is reflected in the US Food and Drug Administration's guidance statement on validation for medical software: "Software testing by itself is not sufficient to establish confidence that the software is fit for its intended use."

Yes, there has been considerable progress in software testing research, and today there are sophisticated testing tools available, but the statement that Dijkstra made, and the position of the FDA on testing, still holds. To get better clarity on the limitations of testing, let us discuss testing from two different perspectives.

Real-world software is complex, and its complexity is rising every year. For example, the size of Windows 3.1 was approximately 4 million LOC in 1990; in 2002, it was 40 million LOC for Windows XP. I don't know the code-base sizes of the latest releases of Linux or Windows, but you can make easy guesses. Many applications I know are more than a million LOC, and their size is increasing every year. The humongous sizes of real-world code-bases make testing extremely difficult. Let me give you a simple example.

A few years back, when I was writing code in Eclipse 3.1, it crashed. I checked its stack trace, which revealed